**DBMS – WEEK 9 REPORT**

**SAREE BOUTIQUE MANAGEMENT SYSTEM**

**Sales and Customer Analytics System**

Scenario

The Saree Boutique Management System manages saree products, customer orders, product categories, and sales details. SQL aggregate functions and analytical queries are used to analyze sales performance, customer purchasing behaviour, best-selling sarees, and category-wise revenue.

Aim

To analyze the sales and customer data of the Saree Boutique Management System using SQL aggregate functions and queries.

Objectives

1. To calculate total orders, total revenue, and average order value.
2. To analyze customer purchases and spending.
3. To identify top customers based on purchase amount.
4. To identify best-selling and least-selling saree products.
5. To perform category-wise sales analysis.
6. To generate business analytics reports for decision-making.

**1. Apply SQL Aggregate Functions**

SQL aggregate functions perform calculations on multiple records and return a single summarized value.

**1.1 COUNT() – Count Total Orders**

```
SQL> SELECT COUNT(*) AS TOTAL_ORDERS
  2  FROM ORDERS;

TOTAL_ORDERS
------------
           2
```

**1.2 SUM() – Calculate Total Sales**

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
  2  FROM ORDERS;

TOTAL_REVENUE
-------------
        40000
```

**1.3 AVG() – Calculate Average Order Value**

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  2  FROM ORDERS;

AVERAGE_ORDER_VALUE
-------------------
              20000
```

**1.4 MIN() – Find Lowest Order Amount**

```
SQL> SELECT MIN(TOTAL_AMOUNT) AS LOWEST_ORDER_AMOUNT
  2  FROM ORDERS;

LOWEST_ORDER_AMOUNT
-------------------
              15000
```

**1.5 MAX() – Find Highest Order Amount**

```
SQL> SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_AMOUNT
  2  FROM ORDERS;

HIGHEST_ORDER_AMOUNT
--------------------
               25000
```

**2. Generate Total Sales Reports**

**Task 1: Write an SQL query to calculate the total number of orders placed.**

```
SQL> SELECT COUNT(*) AS NUMBER_OF_ORDERS_PLACED
  2  FROM ORDERS;

NUMBER_OF_ORDERS_PLACED
-----------------------
                      2
```

**Task 2: Write an SQL query to calculate the total revenue generated from sales.**

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
  2  FROM ORDERS;

TOTAL_REVENUE
-------------
        40000
```

**Task 3: Write an SQL query to calculate the average order value.**

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  2  FROM ORDERS;

AVERAGE_ORDER_VALUE
-------------------
              20000
```

**Task 4: Write an SQL query to find the highest and lowest order amount.**

```
SQL> SELECT
  2      MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_AMOUNT,
  3      MIN(TOTAL_AMOUNT) AS LOWEST_ORDER_AMOUNT
  4  FROM ORDERS;

HIGHEST_ORDER_AMOUNT LOWEST_ORDER_AMOUNT
-------------------- -------------------
               25000               15000
```

**Task 5: Write an SQL query to calculate total sales for a specific period.**

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
  2  FROM ORDERS
  3  WHERE ORDER_DATE BETWEEN
  4        DATE '2026-01-01' AND DATE '2026-12-31';

TOTAL_SALES
-----------
      40000
```

**3. Customer Purchase Analysis**

**Task 1: Write an SQL query to calculate the total number of orders placed by each customer.**

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.CUSTOMER_NAME,
  4      COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  5  FROM ORDERS O
  6  LEFT JOIN ORDERS C
  7      ON C.CUSTOMER_ID = O.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME;

CUSTOME CUSTOMER_NAME                                    TOTAL_ORDERS
------- ------------------------------------------------ ------------
CUS0002 NANDY                                                       1
CUS0001 YAMINI                                                      1
```

**Task 2: Write an SQL query to calculate the total amount spent by each customer.**

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      NVL(SUM(TOTAL_AMOUNT), 0) AS AMOUNT_SPENT
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME;

CUSTOME CUSTOMER_NAME                                         AMOUNT_SPENT
------- -------------------------------------------------- ------------
CUS0002 NANDY                                                       15000
CUS0001 YAMINI                                                      25000
```

**Task 3: Write an SQL query to calculate the average spending per customer.**

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      NVL(AVG(TOTAL_AMOUNT), 0) AS AVERAGE_SPENDING
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME;

CUSTOME CUSTOMER_NAME                                      AVERAGE_SPENDING
------- -------------------------------------------------- ----------------
CUS0002 NANDY                                                         15000
CUS0001 YAMINI                                                        25000
```

**Task 4: Write an SQL query to find customers with the maximum purchase amount.**

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING SUM(TOTAL_AMOUNT) = (
  8      SELECT MAX(TOTAL_SPENDING)
  9      FROM (
 10          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
 11          FROM ORDERS
 12          GROUP BY CUSTOMER_ID, CUSTOMER_NAME
 13      )
 14  );

CUSTOME CUSTOMER_NAME                                      TOTAL_SPENDING
------- -------------------------------------------------- --------------
CUS0001 YAMINI                                                      25000
```

**Task 5: Write an SQL query to find customers with fewer purchases.**

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      COUNT(ORDER_ID) AS TOTAL_PURCHASES
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING COUNT(ORDER_ID) = (
  8      SELECT MIN(TOTAL_PURCHASES)
  9      FROM (
 10          SELECT COUNT(ORDER_ID) AS TOTAL_PURCHASES
 11          FROM ORDERS
 12          GROUP BY CUSTOMER_ID, CUSTOMER_NAME
 13      )
 14  );

CUSTOME CUSTOMER_NAME                                    TOTAL_PURCHASES
------- ---------------------------------------------- ---------------
CUS0002 NANDY                                                        1
CUS0001 YAMINI                                                       1
```

**4. Find Top Customers Based on Purchase Amount**

**Task 1: Write an SQL query to find the top 5 customers based on total spending.**

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      COUNT(ORDER_ID) AS TOTAL_PURCHASES
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING COUNT(ORDER_ID) = (
  8      SELECT MIN(TOTAL_PURCHASES)
  9      FROM (
 10          SELECT COUNT(ORDER_ID) AS TOTAL_PURCHASES
 11          FROM ORDERS
 12          GROUP BY CUSTOMER_ID, CUSTOMER_NAME
 13      )
 14  );

CUSTOME CUSTOMER_NAME                                    TOTAL_PURCHASES
------- ---------------------------------------------- ---------------
CUS0002 NANDY                                                        1
CUS0001 YAMINI                                                       1
```

**Task 2: Write an SQL query to find customers with the maximum number of orders.**

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      COUNT(*) AS NUMBER_OF_ORDERS
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID
  6  HAVING COUNT(*) = (
  7      SELECT MAX(ORDER_COUNT)
  8      FROM (
  9          SELECT COUNT(*) AS ORDER_COUNT
 10          FROM ORDERS
 11          GROUP BY CUSTOMER_ID
 12      )
 13  );

CUSTOME NUMBER_OF_ORDERS
------- ---------------
CUS0002               1
CUS0001               1
```

**Task 3: Write an SQL query to identify frequent customers.**

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      COUNT(ORDER_ID) AS NUMBER_OF_ORDERS
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  ORDER BY NUMBER_OF_ORDERS DESC;

CUSTOME CUSTOMER_NAME                                      NUMBER_OF_ORDERS
------- -------------------------------------------------- ----------------
CUS0001 YAMINI                                                            1
CUS0002 NANDY                                                             1
```

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING SUM(TOTAL_AMOUNT) >= 10000
  8  ORDER BY TOTAL_SPENDING DESC;

CUSTOME CUSTOMER_NAME                                    TOTAL_SPENDING
------- -------------------------------------------------- --------------
CUS0001 YAMINI                                                    25000
CUS0002 NANDY                                                     15000
```

**5. Identify Best-Selling Products**

**Task 1: Write an SQL query to find products with maximum sales quantity.**

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  5  FROM PRODUCT1 P
  6  JOIN ORDER_DETAILS OD
  7      ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  HAVING SUM(OD.QUANTITY) = (
 10      SELECT MAX(TOTAL_QUANTITY)
 11      FROM (
 12          SELECT SUM(QUANTITY) AS TOTAL_QUANTITY
 13          FROM ORDER_DETAILS
 14          GROUP BY PRODUCT_ID
 15      )
 16  );

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P101
Shoe
                  2

P102
T-Shirt
                  2

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P002
Cotton Saree
                  2
```

**Task 2: Write an SQL query to find products generating the highest revenue.**

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      SUM(P.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  5  FROM PRODUCT1 P
  6  JOIN ORDER_DETAILS OD
  7      ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  HAVING SUM(P.PRICE * OD.QUANTITY) = (
 10      SELECT MAX(PRODUCT_REVENUE)
 11      FROM (
 12          SELECT SUM(P1.PRICE * OD1.QUANTITY) AS PRODUCT_REVENUE
 13          FROM PRODUCT1 P1
 14          JOIN ORDER_DETAILS OD1
 15              ON P1.PRODUCT_ID = OD1.PRODUCT_ID
 16          GROUP BY P1.PRODUCT_ID
 17      )
 18  )
 19  ORDER BY TOTAL_REVENUE DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------
P001
Kanchipuram Silk Saree
         8500
```

**Task 3: Write an SQL query to identify least-selling products.**

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      NVL(SUM(OD.QUANTITY), 0) AS TOTAL_QUANTITY_SOLD
  5  FROM PRODUCT1 P
  6  LEFT JOIN ORDER_DETAILS OD
  7      ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  HAVING NVL(SUM(OD.QUANTITY), 0) = (
 10      SELECT MIN(TOTAL_QUANTITY)
 11      FROM (
 12          SELECT
 13              P1.PRODUCT_ID,
 14              NVL(SUM(OD1.QUANTITY), 0) AS TOTAL_QUANTITY
 15          FROM PRODUCT1 P1
 16          LEFT JOIN ORDER_DETAILS OD1
 17              ON P1.PRODUCT_ID = OD1.PRODUCT_ID
 18          GROUP BY P1.PRODUCT_ID
 19      )
 20  )
 21  ORDER BY TOTAL_QUANTITY_SOLD;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P001
Kanchipuram Silk Saree
                  1

P111
Sketch
                  1

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
```

**Task 4: Write an SQL query to identify products requiring promotion.**

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      NVL(SUM(OD.QUANTITY), 0) AS TOTAL_QUANTITY_SOLD
  5  FROM PRODUCT1 P
  6  LEFT JOIN ORDER_DETAILS OD
  7      ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  ORDER BY TOTAL_QUANTITY_SOLD ASC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P111
Sketch
                  1

P001
Kanchipuram Silk Saree
                  1

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P102
T-Shirt
                  2

P101
Shoe

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  2

P002
Cotton Saree
                  2
```

**6. Perform Category-wise Sales Analysis**

**Task 1: Write an SQL query to calculate total sales generated by each category.**

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      SUM(P.PRICE * OD.QUANTITY) AS TOTAL_SALES
  5  FROM CATEGORY1 C
  6  JOIN PRODUCT1 P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  JOIN ORDER_DETAILS OD
  9      ON P.PRODUCT_ID = OD.PRODUCT_ID
 10  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 11  ORDER BY TOTAL_SALES DESC;

CATEGOR CATEGORY_NAME        TOTAL_SALES
------- -------------------- -----------
SC001   Kanchipuram Silk           25500
 SC002  Cotton Sarees               3600
```

**Task 2: Write an SQL query to calculate the number of products sold in each category.**

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      NVL(SUM(OD.QUANTITY), 0) AS TOTAL_PRODUCTS_SOLD
  5  FROM CATEGORY1 C
  6  JOIN PRODUCT1 P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  LEFT JOIN ORDER_DETAILS OD
  9      ON P.PRODUCT_ID = OD.PRODUCT_ID
 10  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 11  ORDER BY TOTAL_PRODUCTS_SOLD DESC;

CATEGOR CATEGORY_NAME        TOTAL_PRODUCTS_SOLD
------- -------------------- -------------------
SC001   Kanchipuram Silk                       3
 SC002  Cotton Sarees                          2
```

**Task 3: Write an SQL query to find the highest revenue-generating category.**

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      SUM(P.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  5  FROM CATEGORY1 C
  6  JOIN PRODUCT1 P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  JOIN ORDER_DETAILS OD
  9      ON P.PRODUCT_ID = OD.PRODUCT_ID
 10  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 11  ORDER BY TOTAL_REVENUE DESC;

CATEGOR CATEGORY_NAME        TOTAL_REVENUE
------- -------------------- -------------
SC001   Kanchipuram Silk             25500
 SC002  Cotton Sarees                 3600
```

**Task 4: Write an SQL query to calculate average sales per category.**

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      NVL(AVG(P.PRICE * OD.QUANTITY), 0) AS AVERAGE_SALES
  5  FROM CATEGORY1 C
  6  JOIN PRODUCT1 P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  JOIN ORDER_DETAILS OD
  9      ON P.PRODUCT_ID = OD.PRODUCT_ID
 10  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 11  ORDER BY AVERAGE_SALES DESC;

CATEGOR CATEGORY_NAME        AVERAGE_SALES
------- -------------------- -------------
SC001   Kanchipuram Silk              8500
 SC002  Cotton Sarees                 3600
```

**7. Generate Business Analytics Reports**

**Report 1: Sales Performance Report**

```
SQL> SELECT
  2      SUM(TOTAL_AMOUNT) AS TOTAL_SALES,
  3      COUNT(ORDER_ID) AS NUMBER_OF_ORDERS,
  4      AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE,
  5      MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_VALUE
  6  FROM ORDERS;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- ------------------- -------------------
      40000                2               20000               25000
```

**Report 2: Customer Analytics Report**

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      COUNT(ORDER_ID) AS NUMBER_OF_ORDERS,
  5      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  6  FROM ORDERS
  7  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  8  ORDER BY TOTAL_SPENDING DESC;

CUSTOME CUSTOMER_NAME                                    NUMBER_OF_ORDERS
------- -------------------------------------------------- ----------------
TOTAL_SPENDING
--------------
CUS0001 YAMINI                                                          1
         25000

CUS0002 NANDY                                                           1
         15000
```

**Report 3: Product Performance Report**

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD,
  5      NVL(SUM(P.PRICE * OD.QUANTITY), 0) AS REVENUE_GENERATED
  6  FROM PRODUCT1 P
  7  LEFT JOIN ORDER_DETAILS OD
  8      ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 10  ORDER BY REVENUE_GENERATED DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
P001
Kanchipuram Silk Saree
            1              8500

P002
Cotton Saree
            2              3600

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------

P102
T-Shirt
            2              1198

P101
Shoe

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
            2               756

P111
Sketch
            1                28
```

**Report 4: Category Analysis Report**

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      NVL(SUM(OD.QUANTITY), 0) AS TOTAL_PRODUCTS_SOLD,
  5      NVL(SUM(P.PRICE * OD.QUANTITY), 0) AS TOTAL_REVENUE
  6  FROM CATEGORY1 C
  7  LEFT JOIN PRODUCT1 P
  8      ON C.CATEGORY_ID = P.CATEGORY_ID
  9  LEFT JOIN ORDER_DETAILS OD
 10      ON P.PRODUCT_ID = OD.PRODUCT_ID
 11  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 12  ORDER BY TOTAL_REVENUE DESC;

CATEGOR CATEGORY_NAME        TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
------- -------------------- ------------------- -------------
SC001   Kanchipuram Silk                       3         25500
 SC002  Cotton Sarees                          2          3600
 SC005   Georgette Sarees                      0             0
SC005   Georgette Sarees                       0             0
SC004   Banarasi Sarees                        0             0
SC004    Banarasi Sarees                       0             0
SC002   Cotton Sarees                          0             0
SC003   Designer Sarees                        0             0

8 rows selected.
```

**Conclusion**

The Saree Boutique Management System uses SQL aggregate functions and analytical queries to evaluate sales, customer purchases, product performance, and category-wise revenue. These reports help the boutique identify popular sarees, understand customer purchasing behaviour, and make better business decisions.